

Strategic Data Science Lab 3: Read-only

Welcome to Lab 3! In this lab, you will work as a finance data professional at **NovaCore Solutions Ltd.** You will use SQL to query, audit and clean financial records in the NovaCore database. You will practise extracting data, identifying data quality issues, joining invoices and payments, and producing finance analysis outputs.

Expected completion time

1–2 hours

Setup: Getting Your Engines Running

Let's get your workspace primed for action! In this section, we'll import the necessary libraries and establish our connection to the finance database. You'll be exploring the existing tables and setting up your own structure for new data as we go.

Why DuckDB?

Think of DuckDB as your personal, high-speed SQL playground. While massive finance datasets usually live on heavyweight servers (like MySQL), we're using DuckDB to keep things fast, serverless, and frustration-free, no complex server configurations, usernames, or passwords required!

A quick tip for your exams: While DuckDB is a fantastic, lightweight way to master your SQL logic, keep in mind that your official ACCA assessments will be based on **MySQL**. Most of what you write here will translate, but note four differences in this lab: MySQL does not support `FULL JOIN` or DuckDB's `INSERT OR REPLACE` syntax, A quick tip for your exams, and MySQL cannot use a `SELECT` alias in `WHERE`. Treat DuckDB as your training ground and MySQL as your final destination!

Every good analyst needs their data. Upload the `novacore_lab3_database.duckdb` file as explained in the README file. Once it's uploaded, we'll have full access to the database and can start hunting for insights.

1. Data Exploration

Now, we need to connect to this database and see the tables inside it

```
%pip install duckdb
```

```
import os
import duckdb

db_file = "novacore_lab3_database.duckdb"

con = duckdb.connect(db_file)

print(con.execute("SHOW TABLES").fetchall())
```

```
[('customers',), ('invoice_payment_summary',), ('invoice_profitability',), ('invoices',), ('payments',), ('products',), ('transactions',)]
```

1.1 Database Overview: What's Under the Hood?

Success! You've connected to the database. We are working with **5 core tables** that act as our primary data sources: customers, products, invoices, payments, and transactions.

You might notice a few other names in the list; these are **Views**.

Tables vs. Views: What's the difference?

- **Tables:** These are the "base" storage containers. They hold the actual raw data (the physical records) that you can INSERT, UPDATE, or DELETE. Think of them as your primary filing cabinets.
- **Views:** These are "virtual" tables. They don't store data themselves; instead, they are saved SQL queries. When you call a view, the database runs the underlying query to display a specific, often pre-filtered or pre-calculated, snapshot of data. They are perfect for simplifying complex reporting without duplicating the underlying records.

1.2 Inspecting the Structure

Run the code below to see the layout of the customers table. Want to explore further? Simply swap 'customers' for another table name to peek under the hood of the entire database!

Please refer to the Database Entity Relationship Diagram (Database ERD) in your file to see the other tables

```
import numpy as np
import pandas as pd
```

```
# Check columns and data types of a table
```

```
con.execute("""  
DESCRIBE customers;  
""").df()
```

Out[4]:

	column_name	column_type	null	key	default	extra
0	customer_id	VARCHAR	NO	PRI	None	None
1	customer_name	VARCHAR	YES	None	None	None
2	industry	VARCHAR	YES	None	None	None
3	customer_segment	VARCHAR	YES	None	None	None
4	region	VARCHAR	YES	None	None	None
5	customer_since_date	DATE	YES	None	None	None
6	credit_rating	VARCHAR	YES	None	None	None
7	credit_limit_CUR	DECIMAL(18,2)	YES	None	None	None

2. SQL Analysis Activities

2.1 Taking Inventory: How Much Data Are We Working With?

Before diving into deep analysis, it's a pro-move to understand the scale of your data. Let's run a quick count to see how many records reside in each of our core tables. This helps you grasp the size of the landscape you're about to query.

```
result = con.execute("""SELECT 'customers' AS table_name, COUNT(*) AS row_count FROM customers  
UNION ALL  
SELECT 'products', COUNT(*) FROM products  
UNION ALL  
SELECT 'invoices', COUNT(*) FROM invoices  
UNION ALL  
SELECT 'payments', COUNT(*) FROM payments  
UNION ALL  
SELECT 'transactions', COUNT(*) FROM transactions;  
""").fetchdf()  
  
print(result)
```

	table_name	row_count
0	customers	2000
1	products	150
2	invoices	18000
3	payments	25000
4	transactions	35000

Look at the volume difference. While our customers and products lists are relatively compact, we are managing 35,000 transactions.

When querying, keep this scale in mind:

- **Aggregations:** For tables like transactions or payments, you will rely heavily on GROUP BY to make sense of the high volume.
- **Performance:** Simple queries will run instantly, but as we move to complex joins and multi-table analysis, efficient filtering becomes crucial to keep your queries running smoothly.

Pro Tip: To keep your queries clean and efficient as you start analysing this data, always remember the usual clause writing order: **SELECT → FROM → WHERE → ORDER BY**.

2.2 Building Your First Queries

Now, let's learn how to extract data with precision. Instead of grabbing everything, we'll build our queries step-by-step.

2.2.1 The "Select All" Approach: Sometimes you need a quick look at the raw data.

```
con.execute("""
SELECT *
FROM customers;
""").df()
```

Out[6]:

	customer_id	customer_name	industry	customer_segment	region	customer_sinc e_date	credit_ rating	credit_limit_ CUR
0	CUST00001	Helio Enterprises Collective	Education	Small Business	West Region	2023-03-19	C	45500.0
1	CUST00002	Summit Industries Ltd	Technology	enterprise	Global Accounts	2021-09-17	C	216400.0
2	CUST00003	Aster Solutions Company	Professional Services	Enterprise	Global Accounts	2017-03-10	C	758800.0
3	CUST00004	Willow Collective Group	Technology	Mid-Market	South Region	2022-01-22	A	188000.0
4	CUST00005	Harbour Solutions Company	Retail	Enterprise	Central Region	2022-04-20	A	517500.0
...	...	...	...	...	...	...	...	...
1995	CUST01996	Willow Services Collective	Healthcare	Small Business	North Region	2022-12-01	A	59000.0
1996	CUST01997	Juniper Holdings Collective	Logistics	Small Business	Central Region	2017-05-23	B	44400.0
1997	CUST01998	Silverline Works Company	Financial Services	Enterprise	West Region	2023-05-11	A	416200.0
1998	CUST01999	Lumina Holdings Company	Logistics	Mid-Market	West Region	2021-09-12	B	101800.0
1999	CUST02000	Aster Collective Company	Technology	Small Business	Global Accounts	2018-09-25	D	58100.0

2000 rows × 8 columns

2.2.2 Selecting Specific Columns: In professional environments, we avoid **SELECT *** because it's inefficient and messy. It's better practice to request only the specific columns you need.

```

con.execute("""
SELECT
    customer_id,
    customer_name,
    industry,
    region
FROM customers;
""").df()

```

Out[7]:

	customer_id	customer_name	industry	region
0	CUST00001	Helio Enterprises Collective	Education	West Region
1	CUST00002	Summit Industries Ltd	Technology	Global Accounts
2	CUST00003	Aster Solutions Company	Professional Services	Global Accounts
3	CUST00004	Willow Collective Group	Technology	South Region
4	CUST00005	Harbour Solutions Company	Retail	Central Region
...	...	...	...	...
1995	CUST01996	Willow Services Collective	Healthcare	North Region
1996	CUST01997	Juniper Holdings Collective	Logistics	Central Region
1997	CUST01998	Silverline Works Company	Financial Services	West Region
1998	CUST01999	Lumina Holdings Company	Logistics	West Region
1999	CUST02000	Aster Collective Company	Technology	Global Accounts

2000 rows × 4 columns

2.2.3 Adding Filters with WHERE: This is where the magic happens. We narrow down our dataset to focus only on records that meet our criteria.

```
con.execute("""
SELECT
    customer_id,
    customer_name,
    customer_segment,
    credit_limit_CUR
FROM customers
WHERE customer_segment = 'Enterprise';
""").df()
```

Out[8]:

	customer_id	customer_name	customer_segment	credit_limit_CUR
0	CUST00003	Aster Solutions Company	Enterprise	758800.0
1	CUST00005	Harbour Solutions Company	Enterprise	517500.0
2	CUST00006	Cobalt Industries Collective	Enterprise	221200.0
3	CUST00007	Meridian Holdings Company	Enterprise	304300.0
4	CUST00010	Vector Services Collective	Enterprise	453300.0
...	...	...	...	...
367	CUST01972	Redwood Enterprises Ltd	Enterprise	242700.0
368	CUST01976	Harbour Works Collective	Enterprise	558800.0
369	CUST01984	Nimbus Holdings Ltd	Enterprise	521800.0
370	CUST01988	Pinnacle Ventures Company	Enterprise	171300.0
371	CUST01998	Silverline Works Company	Enterprise	416200.0

372 rows × 4 columns

Think of it like a funnel. SELECT defines the columns (the "what"), FROM identifies the source (the "where"), and WHERE acts as your filter (the "which specific records"). By combining these, you transform a massive, unmanageable table into a clean, actionable dataset.

2.2.4 Sorting Your Insights: Now that you've mastered selecting and filtering, let's add the final piece: **Ordering**.

When dealing with financial limits or transaction amounts, simply seeing the data isn't enough; you need to see the outliers. The ORDER BY clause allows you to rank your data, helping you instantly spot your highest credit exposures.

```
con.execute("""
SELECT
    customer_name,
    customer_segment,
    credit_limit_CUR
FROM customers
ORDER BY credit_limit_CUR DESC;
""").df()
```

Out[9]:

	customer_name	customer_segment	credit_limit_CUR
0	Aster Partners Company	Enterprise	798600.0
1	Nimbus Solutions Group	Enterprise	797900.0
2	Ion Networks Ltd	Enterprise	797700.0
3	Summit Systems Collective	Enterprise	797300.0
4	Cobalt Networks Group	Enterprise	796100.0
...	...	...	...
1995	Crescent Alliance Collective	Small Business	10300.0
1996	Ion Works Collective	Small Business	10300.0
1997	Granite Collective Ltd	Small Business	10200.0
1998	Ion Operations Company	Small Business	10100.0
1999	Vector Services Ltd	Small Business	10100.0

2000 rows × 3 columns

By adding DESC (Descending), we put the highest values at the top of our list. This is a staple in financial reporting—whether you're sorting by overdue payments, largest transaction values, or top-performing products.

You have now completed the usual clause writing order: **SELECT → FROM → WHERE → ORDER BY**.

2.3 Building Our Own Infrastructure: The "New Request" Challenge

You've mastered querying existing data, but what happens when the Finance Director asks for something new?

Your boss just dropped a request: *"We need a way to track manual payment adjustments that aren't captured in the main invoices table."* To handle this, we can't just keep adding rows to a spreadsheet; we need to build a robust, structured table within our database.

This is where **Data Definition Language (DDL)** comes into play. It's how we act as architects, not just explorers.

Why DDL Matters in Finance

Before defining our table, remember that precision is everything. We use:

- **Primary Keys:** To ensure every adjustment has a unique identifier, no duplicates allowed.
- **Foreign Keys:** To link our new table to the `invoices` table, maintaining "referential integrity" (so we can't create an adjustment for an invoice that doesn't exist!).
- **DECIMAL Types:** We avoid `FLOAT` for money. `DECIMAL(10,2)` ensures we don't run into rounding errors that could cause massive discrepancies in year-end audits.

A colleague proposes using `FLOAT` for `adjustment_amount`. Would you approve? Explain one possible consequence for financial reconciliation and recommend a suitable data type.

Let's build the `payment_adjustments` table:

```
con.execute("""
CREATE TABLE IF NOT EXISTS payment_adjustments (
    adjustment_id INTEGER PRIMARY KEY,
    invoice_id VARCHAR,
    adjustment_date DATE,
    adjustment_amount DECIMAL(10,2),
    reason VARCHAR,
    FOREIGN KEY (invoice_id) REFERENCES invoices(invoice_id)
);
""")
```

Out[10]:

```
<_duckdb.DuckDBPyConnection at 0x7fda96bddeb0>
```

Now that we've built our infrastructure, let's populate it: run this command to insert your initial adjustments into the database so we have some real evidence to analyse.

```
con.execute("""
INSERT or REPLACE INTO payment_adjustments VALUES
(1, 'INV000000001', DATE '2025-01-15', 250.50, 'Credit note'),
(2, 'INV000000002', DATE '2025-02-10', -100.00, 'Correction'),
(3, 'INV000000003', DATE '2025-03-05', 75.25, 'Adjustment');
""")
```

Out[11]:

```
<_duckdb.DuckDBPyConnection at 0x7fda96bddeb0>
```

```
con.execute("""
SELECT *
FROM payment_adjustments;
""").df()
```

Out[12]:

	adjustment_id	invoice_id	adjustment_date	adjustment_amount	reason
0	2	INV000000002	2025-02-10	-100.00	Correction
1	1	INV000000001	2025-01-15	250.50	Credit note
2	3	INV000000003	2025-03-05	75.25	Adjustment

Note: This table is added to the existing NovaCore database. The original tables (customers, invoices, payments, etc.) remain unchanged.

```
print(con.execute("SHOW TABLES").fetchall())
```

```
[('customers',), ('invoice_payment_summary',), ('invoice_profitability',), ('invoices',), ('payment_adjustments',), ('payments',), ('products',), ('transactions',)]
```

2.4 The Data Sanitisation Station: Cleaning Before Querying

Before you can produce accurate financial reports, you need to ensure your data is reliable. In the real world, data often arrives messy, fragmented, or duplicated—and running analysis on "dirty" data is a recipe for disaster.

In this stage, we'll tackle the three most common culprits of poor data quality:

- **Missing Values:** We'll use `COALESCE()` to fill gaps for display purposes, ensuring our reports aren't missing key figures.
- **Duplicates:** We'll learn how to identify and prune redundant entries that could artificially inflate transaction volumes or customer counts.

- **Standardisation:** We'll use functions like `TRIM()`, `UPPER()`, and `LOWER()` to fix inconsistent formatting—turning messy inputs like " enterprise" and "ENTERPRISE " into a single, uniform standard.

Important Note: These techniques are non-destructive. They help you clean and standardise your query results, leaving your original "source of truth" in the database safely untouched. In finance and audit work, analysts usually preserve the original data and create cleaned outputs, views or reports to maintain an audit trail.

Let's start by looking at how to handle those pesky NULL values.

2.4.1 Check for missing values in customer data

Example: identify customers with missing credit ratings.

```
con.execute("""
SELECT
    COUNT(*) AS missing_credit_rating
FROM customers
WHERE credit_rating IS NULL;
""").df()
```

Out[14]:

missing_credit_rating	
0	20

You just discovered 20 missing credit ratings! This is a classic "real-world" scenario. In finance, a NULL isn't just a blank space—it's a data point that tells a story. Is the credit rating missing because the customer is new, or is it an error in our system?

Before we do anything, knowing *where* the gaps are is step one. Now that you've identified the scale of the missing data (20 records), we can proceed to clean it up for our reporting using `COALESCE()`.

2.4.2 Use `COALESCE` for reporting labels

`COALESCE()` returns the first non-NULL value, or NULL if all values are NULL. An empty string is not NULL. Here it displays Not Available for missing ratings without changing the stored data.

```
con.execute("""
SELECT
    customer_id,
    customer_name,
    COALESCE(credit_rating, 'Not Available') AS reporting_credit_rating
FROM customers;
""").df()
```

Out[15]:

	customer_id	customer_name	reporting_credit_rating
0	CUST00001	Helio Enterprises Collective	C
1	CUST00002	Summit Industries Ltd	C
2	CUST00003	Aster Solutions Company	C
3	CUST00004	Willow Collective Group	A
4	CUST00005	Harbour Solutions Company	A
...	...	...	...
1995	CUST01996	Willow Services Collective	A
1996	CUST01997	Juniper Holdings Collective	B
1997	CUST01998	Silverline Works Company	A
1998	CUST01999	Lumina Holdings Company	B
1999	CUST02000	Aster Collective Company	D

2000 rows x 3 columns

It's likely that Not Available is indeed there, but it's hidden in the middle of those 2,000 rows! The output you are seeing is a preview that shows the first few rows and the last few rows, skipping the ones in between.

If some of your customers have NULL credit ratings, they will definitely be displaying as Not Available in the reporting_credit_rating column. To check the missing source ratings, use WHERE credit_rating IS NULL.

Your turn: Try checking another nullable column, such as region in the customers table.

2.4.3 Duplicate Records and Text Standardisation Now that we've handled missing values, it's time to tackle the "dirty data" that can sabotage your analysis. Duplicate records and inconsistent formatting are silent killers in finance. they lead to overstated revenue, bloated customer lists, and inaccurate receivables balances.

A duplicate isn't always a carbon copy. Sometimes it's a transaction recorded twice with a slightly different timestamp. We use GROUP BY and HAVING to find records that appear more than once.

```
con.execute("""
SELECT
    invoice_id,
    COUNT(*) AS record_count
FROM invoices
GROUP BY invoice_id
HAVING COUNT(*) > 1;
""").df()
```

Out[16]:

invoice_id	record_count
------------	--------------

```
con.execute("""
SELECT *
FROM invoices
WHERE invoice_id IN (
    SELECT invoice_id
    FROM invoices
    GROUP BY invoice_id
    HAVING COUNT(*) > 1
);
""").df()
```

Out[17]:

invoice_id	customer_id	product_id	invoice_date	due_date	quantity	invoice_amount_CUR	payment_status
------------	-------------	------------	--------------	----------	----------	--------------------	----------------

Ever seen a database where 'Enterprise', 'enterprise ', and 'ENTERPRISE' are treated as three different segments? That ruins your reporting. We use TRIM(), UPPER(), and LOWER() to ensure consistency.

Let's investigate this text inconsistency issue in our customer data. A powerful first step is to isolate and view all the unique values within the customer_segment column. This will help us spot any variations in casing or spacing that need to be standardised.

```
con.execute("""
SELECT DISTINCT customer_segment
FROM customers;
""").df()
```

Out[18]:

	customer_segment
0	Enterprise
1	Mid-Market
2	enterprise
3	mid-market
4	Small Business
5	small business

- **On Duplicates:** The query found no duplicate invoice_id values. This confirms invoice IDs are unique, but it does not establish that every table or transaction is free of duplicates.
- **On Standardisation:** This is where we have our work cut out for us. You've correctly identified that our customer_segment column contains "duplicate-like" issues caused by case inconsistency (Enterprise vs enterprise). While these aren't technical duplicates (the database sees them as different strings), they are analytical duplicates that will break your reports if left uncleaned.

Why this distinction matters? Duplicate Records are technical errors (e.g., the same invoice appearing twice) that must be removed. Standardisation Issues are formatting inconsistencies (e.g., inconsistent casing) that must be harmonised.

Now that we've checked invoice IDs, let's get those segments cleaned up so your data analysis is rock-solid!

```
con.execute("""
SELECT
    customer_id,
    customer_segment,
    UPPER(TRIM(customer_segment)) AS cleaned_segment
FROM customers
WHERE customer_segment <> UPPER(TRIM(customer_segment));
""").df()
```

Out[19]:

	customer_id	customer_segment	cleaned_segment
0	CUST00001	Small Business	SMALL BUSINESS
1	CUST00002	enterprise	ENTERPRISE
2	CUST00003	Enterprise	ENTERPRISE
3	CUST00004	Mid-Market	MID-MARKET
4	CUST00005	Enterprise	ENTERPRISE
...	...	...	...
1995	CUST01996	Small Business	SMALL BUSINESS
1996	CUST01997	Small Business	SMALL BUSINESS
1997	CUST01998	Enterprise	ENTERPRISE
1998	CUST01999	Mid-Market	MID-MARKET
1999	CUST02000	Small Business	SMALL BUSINESS

2000 rows x 3 columns

The query returned all 2,000 records because the chosen reporting standard is uppercase. This does not mean that every record contains a data-quality error. However, the distinct values identified earlier confirm that inconsistent capitalisation exists and should be standardised for reliable analysis.

The Takeaway: By running this comparison, you've not only identified the scope of the problem but also verified the fix. You are now ready to perform accurate, group-based analysis without those pesky formatting errors splitting your data into multiple pieces.

2.5 The Power of Joins: Visualising the Gaps

Joins are how we connect different tables. To really understand them, we need to look at what happens when data exists in one table but not the other.

Think of our invoices and payments tables:

- **LEFT JOIN:** Keeps all invoices, even if no payment exists (the "Invoices without a recorded payment" view).
- **RIGHT JOIN:** Keeps all payments, including any without a matching invoice. In this database, the payment foreign key prevents such unmatched records.
- **FULL JOIN:** Keeps everything from both sides, showing matches and any gaps in either direction. This dataset contains no unmatched payments. The foreign key rejects invalid non-NULL invoice IDs.

2.5.1. LEFT JOIN (The "Invoices-First" View): This shows every invoice. If there is no corresponding payment, it returns NULL for the payment details.

```
con.execute("""
SELECT i.invoice_id, p.payment_id
FROM invoices i
LEFT JOIN payments p ON i.invoice_id = p.invoice_id
ORDER BY p.payment_id NULLS FIRST, i.invoice_id
LIMIT 10;
""").df()
```

Out[20]:

	invoice_id	payment_id
0	INV00000001	None
1	INV00000010	None
2	INV00000011	None
3	INV00000018	None
4	INV00000021	None
5	INV00000044	None
6	INV00000047	None
7	INV00000048	None
8	INV00000061	None
9	INV00000064	None

```
# This will show you exactly which invoices have no matching payment
# (based on your filter criteria)
con.execute("""
SELECT i.invoice_id, p.payment_id
FROM invoices i
LEFT JOIN payments p ON i.invoice_id = p.invoice_id
WHERE p.payment_id IS NULL;
""").df()
```

Out[21]:

	invoice_id	payment_id
0	INV00000144	None
1	INV00000234	None
2	INV00000317	None
3	INV00000471	None
4	INV00000640	None
...	...	...
2442	INV00017713	None
2443	INV00017719	None
2444	INV00017837	None
2445	INV00017861	None
2446	INV00017940	None

2447 rows × 2 columns

2.5.2 RIGHT JOIN (The "Payments-First" View): This shows every payment. If a payment exists without a linked invoice (a data anomaly), it still shows up here.

```
con.execute("""
SELECT i.invoice_id, p.payment_id
FROM invoices i
RIGHT JOIN payments p ON i.invoice_id = p.invoice_id
ORDER BY p.payment_id, i.invoice_id
LIMIT 10;
""").df()
```

Out[22]:

	invoice_id	payment_id
0	INV00000002	PAY00000001
1	INV00000003	PAY00000002
2	INV00000004	PAY00000003
3	INV00000004	PAY00000004
4	INV00000005	PAY00000005
5	INV00000006	PAY00000006
6	INV00000007	PAY00000007
7	INV00000008	PAY00000008
8	INV00000009	PAY00000009
9	INV00000012	PAY00000010

```
con.execute("""
SELECT p.payment_id, p.invoice_id
FROM payments p
LEFT JOIN invoices i ON p.invoice_id = i.invoice_id
WHERE i.invoice_id IS NULL;
""").df()
```

Out[23]:

payment_id	invoice_id
------------	------------

This returns no rows: every payment is linked to an existing invoice because `payments.invoice_id` has a foreign key. An unmatched payment would require investigation if the constraint were absent.

2.5.3. FULL JOIN (The "Everything" View): This captures the complete relationship. It can show invoices without payments and payments without invoices. In this database, the foreign key prevents unmatched payments.

```
con.execute("""
SELECT i.invoice_id, p.payment_id
FROM invoices i
FULL JOIN payments p ON i.invoice_id = p.invoice_id
ORDER BY p.payment_id NULLS FIRST, i.invoice_id
LIMIT 10;
""").df()
```

Out[24]:

	invoice_id	payment_id
0	INV00000001	None
1	INV00000010	None
2	INV00000011	None
3	INV00000018	None
4	INV00000021	None
5	INV00000044	None
6	INV00000047	None
7	INV00000048	None
8	INV00000061	None
9	INV00000064	None

When working with joins, you need two different viewing modes:

- **The Spot Check (LIMIT 10):** Use this during the initial design phase to verify your join logic and ensure the columns you need are present.
- **The Audit Report (WHERE ... IS NULL):** Use this to find the "orphans", the records that didn't match. In finance, this is often the most important query, as it can reveal invoices without recorded payments. This database's payment foreign key prevents payments without a matching invoice.

2.6 Simple Derived Features and Outlier Discussion

Finance professionals often create KPIs and calculated fields from raw data.

Examples:

- Gross invoice value
- Discount amount
- Net sales
- Profit
- Outstanding balance

Large values should be investigated. An unusually large invoice may be:

- A data error
- A genuine high-value transaction

```
# 1. Aggregated Derived Features
# This gives you a high-level summary of your financial portfolio
con.execute("""
SELECT
    COUNT(invoice_id) AS total_invoice_count,
    ROUND(SUM(invoice_amount_CUR), 2) AS total_gross_value,
    ROUND(SUM(invoice_amount_CUR * 0.05), 2) AS total_discount_amount,
    ROUND(SUM(invoice_amount_CUR * 0.95), 2) AS total_net_sales,
    ROUND(AVG(invoice_amount_CUR), 2) AS average_invoice_value,
    MIN(invoice_amount_CUR) AS lowest_invoice,
    MAX(invoice_amount_CUR) AS highest_invoice
FROM invoices;
""").df()
```

Out[25]:

	total_invoice_count	total_gross_value	total_discount_amount	total_net_sales	average_invoice_value	lowest_invoice	highest_invoice
0	18000	410031999.0	20501599.95	3.895304e+08	22779.56	150.0	9999999.0

What's new in this analysis: Summarisation: Instead of showing individual invoices, we now see the total business performance across the entire dataset.

ROUND(..., 2): Essential for finance—it ensures your totals are displayed in clean currency format (two decimal places).

Aggregate Functions: * COUNT() tells you your total volume.

SUM() gives you your top-line revenue figures.

AVG(), MIN(), and MAX() provide the context needed to understand your invoice distribution (the "spread").

Analyst Insight: This summary query is usually the "Header" of your monthly financial report. By comparing your MIN and MAX values, you can quickly see the range of your business...if the gap is massive, it's a perfect trigger to investigate those outliers we talked about!

```
#2. Calculate outstanding balance
con.execute("""
SELECT
    i.invoice_id,
    invoice_amount_CUR,
    COALESCE(SUM(payment_amount_CUR),0) AS total_paid,
    invoice_amount_CUR - COALESCE(SUM(payment_amount_CUR),0)
        AS outstanding_balance
FROM invoices i
LEFT JOIN payments p
    ON i.invoice_id = p.invoice_id
GROUP BY
    i.invoice_id,
    invoice_amount_CUR
LIMIT 10;
""").df()
```

Out[26]:

	invoice_id	invoice_amount_CUR	total_paid	outstanding_balance
0	INV000000002	1440.0	1440.0	0.0
1	INV000000026	13820.0	14050.0	-230.0
2	INV000000027	3060.0	3060.0	0.0
3	INV000000028	880.0	260.0	620.0
4	INV000000059	21480.0	21480.0	0.0
5	INV000000063	3950.0	1360.0	2590.0
6	INV000000071	98290.0	101430.0	-3140.0
7	INV000000072	210.0	210.0	0.0
8	INV000000084	43270.0	19010.0	24260.0
9	INV000000088	46970.0	46970.0	0.0

Analyst Insight: Think of these two queries as your **Finance Dashboard's dual-pane view**:

- **The Summary (Query #1):** Provides your high-level "Bird's-Eye View" for trend analysis and monthly reporting.
- **The Detail (Query #2):** Provides your "Operational View" for individual account management and reconciliation.

You need the summary to understand the business, but you need the detail to actually manage the cash. Every effective financial report pairs these two perspectives to ensure you never lose track of the big picture or the individual transaction!

```
#3. Identify unusually large invoices
con.execute("""
SELECT
    invoice_id,
    customer_id,
    invoice_amount_CUR
FROM invoices
ORDER BY invoice_amount_CUR DESC
LIMIT 10;
""").df()
```

Out[27]:

	invoice_id	customer_id	invoice_amount_CUR
0	INV00011587	CUST00242	9999999.0
1	INV00002767	CUST01364	950000.0
2	INV00008525	CUST01869	950000.0
3	INV00009345	CUST00959	950000.0
4	INV00010952	CUST00194	950000.0
5	INV00014479	CUST00675	938270.0
6	INV00015435	CUST01438	937740.0
7	INV00017374	CUST00675	917820.0
8	INV00001205	CUST00579	741940.0
9	INV00002935	CUST00579	686230.0

Spotting the Anomalies: Outlier Analysis

Even with your KPIs calculated, your job as an auditor isn't done. The final step is to check for **outliers**, unusually large transactions that can skew your averages and mislead your reporting.

This query helps you identify the "tails" of your distribution. If your AVG (from query #1) is 500, but these top 10 invoices are all 50,000+, you know that a few massive deals are driving your entire narrative.

Why this matters:

- **Error Detection:** Is that \$1,000,000 invoice a genuine sale, or a data entry mistake (e.g., an extra zero)?
- **Risk Management:** High-value invoices represent concentration risk; if one of these clients doesn't pay, it significantly impacts your cash flow.

Analyst Insight: Always investigate your extremes. If you find a massive invoice, cross-reference it with the customer_id and the outstanding_balance query we ran earlier. If a large invoice is also overdue, that is your **#1 priority for follow-up!**

3. Your Turn: Apply What You Learned

Complete the following activities using the NovaCore finance database.

Task 1: Challenge a financial query

A colleague uses this query to report the total value of **all invoices**:

```
SELECT SUM(i.invoice_amount_CUR) AS total_invoiced
FROM invoices i
JOIN payments p ON i.invoice_id = p.invoice_id;
```

Would you trust the result? Explain how invoices with no payments and invoices with several payments affect it. Correct the query and justify your change.

Author check: The join excludes invoices without payments and repeats amounts for invoices with multiple payments. Summing invoices directly gives CUR 410,031,999.

Task 2: Finance KPI

Calculate the **total invoiced amount by customer segment**, using a consistent segment label (UPPER(TRIM(customer_segment))).

Skills: SUM() | GROUP BY

Task 3: Data Quality

Identify customer segment values that need standardisation.

Create a cleaned version using:

- TRIM()
- UPPER() or LOWER()

Skills: Text standardisation functions

Task 4: DDL Practice

Create a new table called customer_notes.

The table should include:

- note_id → Primary Key
- customer_id → Foreign Key linked to customers
- note_date → DATE
- note_text → VARCHAR

Insert at least two records and display the table contents.

Skills: CREATE TABLE | PRIMARY KEY | FOREIGN KEY | DATE | INSERT

Reflection Questions

1. **The "Missing Link" Audit:** If an invoice has no linked payment, what would you check before deciding whether it needs follow-up? Could it be not yet due or a recording error?
2. **The KPI Trap:** We calculated "Net Sales" using a flat 5% discount. Why might this simple model be dangerous if your actual client discounts vary wildly?
3. **Outlier Strategy:** If you find a massive invoice, what steps would you take to confirm it's a "win" rather than a data entry mistake before reporting it to the CFO?
4. **Choosing a data architecture:** Suppose NovaCore's invoice, payment and customer data were held in separate departmental systems. For recurring financial reporting, would you recommend a data warehouse or a data lake? Justify your choice and identify one data-quality control.

4. MISSION ACCOMPLISHED!

Are you tired? Because you should be—that was a serious workout! Seriously, grab a drink, shake out those wrists, and take a breather. You've earned it.

You've officially survived Lab 3. You've gone from "Data Novice" to "Finance Detective."

In this session, you successfully:

- **Played Auditor:** Identified missing ratings and displayed a reporting label with `COALESCE()`.
- **Cleaned the Mess:** Standardised inconsistent labels so your database doesn't think 'Enterprise' and 'enterprise' are from different planets.
- **Reconciled the Books:** Mastered the art of `LEFT JOIN` to identify invoices without recorded payments.
- **KPI Ninja:** Turned raw numbers into actionable financial insights using Aggregations.
- **Table Architect:** Built your own `payment_adjustments` table from scratch.

The Road Ahead... You've conquered the basics and the middle-ground, but don't get too comfortable.

Great work today!